

# Project Sentinel - Detailed Implementation Plan

Generated for Project Sentinel pre-coding readiness package.

## Project Sentinel - Detailed Implementation Plan

### Phase 0 - Repository normalization and coding preparation

Objective: make the repository ready for code while preserving the fixed hierarchy.

Deliverables:

- ``pyproject.toml``
- ``gitignore``
- ``README.md`` coding start section
- ``src/sentinel/__init__.py``
- ``tests/unit/``
- ``tests/integration/``
- ``scripts/dev_setup.sh``
- ``tools/validate_repo.py``

Exit criteria:

- Repository imports successfully.
- Unit test runner works.
- Formatting and linting baseline is configured.

### Phase 1 - C-001 Foundation Runtime

Build first:

- Sentinel Kernel
- Engine lifecycle protocol
- Universal Object Model base classes
- Event envelope
- In-memory Event Bus
- Evidence record base
- Policy stub
- Trust stub
- Plugin registration contract

Recommended module layout:

```
src/sentinel/  
  kernel/  
    lifecycle.py  
    engine.py
```

```
registry.py
runtime.py
object_model/
  base.py
  identity.py
  asset.py
  event.py
  evidence.py
event_bus/
  bus.py
  envelope.py
  handlers.py
evidence/
  records.py
  store.py
policy/
  engine.py
  rules.py
trust/
  score.py
  engine.py
plugins/
  contract.py
config/
  settings.py
```

Exit criteria:

- A Sentinel runtime can start and stop.
- Engines can register with the Kernel.
- Events can be published and consumed.
- Objects have stable IDs, type names, timestamps, and metadata.
- Evidence can be attached to an event or finding.
- Tests pass.

## Phase 2 - C-002 Core services

Build:

- Persistent event store abstraction
- Evidence store abstraction
- Policy evaluation result model
- Trust score model
- Basic audit log
- Configuration loader
- Error model
- Correlation ID propagation

Exit criteria:

- Events and evidence can be serialized and reloaded.
- Policy and trust engines produce deterministic outputs.
- Audit logs are JSON structured.

## **Phase 3 - C-003 Knowledge Graph and Mission Context**

Build:

- Graph node and edge models
- Relationship types
- Mission object model
- Mission criticality scoring
- Graph adapter interface
- In-memory graph backend

Exit criteria:

- Assets, identities, and events can be linked.
- Mission criticality can influence prioritization.

## **Phase 4 - C-004 Workflow and Automation Runtime**

Build:

- Workflow definition model
- Workflow state machine
- Task execution interface
- Remediation recommendation model
- Approval gate model
- Workflow event publishing

Exit criteria:

- A remediation workflow can be created, run, paused, approved, and completed.

## **Phase 5 - C-005 Asset and identity engines**

Build:

- Asset inventory engine
- Identity inventory engine
- ISPM engine foundation
- Machine identity governance foundation
- Connector interface
- Mock connectors

Exit criteria:

- Mock assets and identities can be discovered, normalized, scored, and published.

## **Phase 6 - C-006 Detection, vulnerability, threat intelligence, and exposure**

Build:

- Detection rule model
- Finding model
- Vulnerability object model
- Exposure object model
- Threat intel object model
- Correlation engine skeleton

Exit criteria:

- A vulnerability and an identity risk can be correlated into a finding.

## **Phase 7 - C-007 Incident, forensics, compliance, risk, assurance**

Build:

- Incident case model
- Evidence preservation workflow
- Compliance mapping model
- Risk quantification model
- Control validation model

Exit criteria:

- A finding can become a case with evidence, compliance impact, and risk score.

## **Phase 8 - C-008 AI, privacy, data protection, and governance**

Build:

- AI model governance object model
- Privacy classification model
- Data sovereignty tags
- Data handling policy hooks

Exit criteria:

- Data objects can carry privacy/security classification and policy constraints.

## **Phase 9 - C-009 API, SDK, CLI, plugins**

Build:

- REST API skeleton
- SDK package
- CLI commands
- Plugin loader
- Developer examples

Exit criteria:

- Developers can query objects, publish events, and install a simple plugin.

## Phase 10 - C-010 Deployment and operations

Build:

- Dockerfile
- docker-compose for dev
- systemd service example
- database migration scripts
- health endpoints
- backup/restore scripts
- release packaging

Exit criteria:

- Sentinel can run on a VPS as a service with logs, backups, and health checks.

## First coding sprint

The first sprint should be intentionally small:

1. Create ``pyproject.toml``.
2. Create the ``sentinel`` Python package.
3. Implement ``SentinelObject``.
4. Implement ``SentinelEvent``.
5. Implement in-memory ``EventBus``.
6. Implement ``SentinelKernel``.
7. Write unit tests.
8. Add a simple example in ``examples/hello_sentinel.py``.

Definition of done:

- ``pytest`` passes.
- Example runs from command line.
- Code is typed and documented.
- No engine-specific complexity is introduced yet.